Entropy-Aware and UCB-Driven Coverage Closure for Hardware Verification

Bangliang Chen[1], Xiexi Xiong[1], Yiqiu Liu[1,2,*]
1. South China University of Technology, Guangzhou, China
2. The Engineering Center for DTCO of Integrated Circuits, Ministry of Education, China
* Corresponding author: yiqiuliu@scut.edu.cn

***Abstract*—Coverage closure remains a bottleneck in constrained-random hardware verification because late-stage simulation often repeatedly exercises behaviors that have already been covered. This paper presents an entropy-aware and upper-confidence-bound (UCB)-driven framework for coverage-directed simulation. Functional coverage bins are grouped into semantic clusters and assigned static importance scores based on rarity, entropy-shaped uncertainty, diversity, structural complexity, and cluster size. During online execution, these priorities are updated according to the remaining coverage opportunity. A continuous-refill two-level UCB scheduler selects a target cluster and a corresponding test sequence, while runtime-aware scoring accounts for heterogeneous simulation costs. Experiments on OpenTitan AON Timer and a 19,919-bin industrial IP demonstrate improved late-stage coverage growth over random selection and scheduler ablations under comparable simulation budgets. On the industrial IP, the full method reaches 60.16% raw coverage, compared with 57.87% for No Info and 55.09% for Random Selection.**

***Index Terms*—coverage closure, hardware verification, entropy, upper confidence bound**

I. Introduction

Functional coverage is a central progress metric in constrained-random hardware verification. Modern verification environments can generate large numbers of legal tests under user-defined constraints, but they still require feedback mechanisms to decide which parts of the stimulus space deserve more simulation effort [1]. Coverage-directed generation addresses this problem by using coverage results to guide future tests, and prior work has explored learning-based models for this feedback loop [2], [3]. Despite these advances, coverage closure remains difficult in the late stage: most easy bins have already been exercised, while the remaining bins are sparse, correlated, and unevenly distributed across coverpoints and cross-coverage structures. Recent work on last-mile coverage further highlights that these hard-to-reach bins remain a practical bottleneck even in automated flows [4].

A key limitation of coverage metrics based only on newly covered-bin counts is that all new bins receive equal credit. In practice, rarely exercised, structurally complex, or weakly correlated bins may represent less-explored regions of the verification space.

We cluster bins using semantic name features, score clusters by rarity, uncertainty, diversity, size, and structural complexity, and update their importance online. A two-level UCB policy selects target clusters and sequences, with a runtime penalty for simulation cost [5], [6].

The main contributions are: (1) an information-based scoring model for prioritizing functional-coverage clusters; (2) a component-aware clustering procedure that converts raw coverage bins into schedulable target regions; and (3) a continuous-refill two-level UCB scheduler that jointly considers target priority, historical sequence gain and simulation runtime.

II. Related Work

Coverage-directed generation uses simulation feedback to bias stimuli or predict promising tests [1]–[3], [7], [8]. We instead cluster bins, assign information-aware utility, and schedule clusters and sequences online.

Regression test selection, novelty detection, and learning-based verification reduce simulation cost by selecting representative or promising tests [4], [9]–[16]. Huang et al. tune high-level constrained-random test parameters using coverage-guided random search and Bayesian optimization [17], while Masamba et al. use supervised learning to prioritize tests likely to exercise remaining coverage holes [18].

RTL fuzzing systems such as RFUZZ and DirectFuzz use coverage feedback for directed exploration [19], [20], and MABFuzz applies bandit policies to hardware fuzzing [21]. Most closely related to our scheduler, Dimitrakopoulos et al. formulate autonomous test-sequence application as a multi-armed bandit problem [22]. Our method instead uses information-theoretic cluster priors and two-level UCB to first select a target coverage cluster and then a target-conditioned sequence while accounting for simulation runtime.

III. Background and Motivation

A. Coverage Closure

Let $\mathcal{B}$ denote the set of functional coverage bins and let $h_b(t)$ be the hit count of bin b after simulation t. Standard coverage closure treats all uncovered bins equally, ignoring that some bins are structurally more valuable and some are empirically rarer under existing tests.

B. Rarity and Information of Bins

Following the self-information view from information theory [5], a rare event carries larger information. We first convert the historical coverage data into a normalized relative

hit frequency $f_b = \bar{h}_b / \sum_{b' \in \mathcal{B}} \bar{h}_{b'}$, where $\bar{h}_b$ is its average initial-corpus hit count. The bin-level rarity, expressed as self-information, is $I(b) = -\log(f_b)$, where a numerical floor handles zero frequency. The static cluster-level rarity is the average $I_{\text{avg}}(C) = |C|^{-1} \sum_{b \in C} I(b)$. Thus, $I(b)$ and $I_{\text{avg}}(C)$ quantify rarity; they are not online rewards. The self-information score is used to construct a cluster-level scheduling prior rather than as the online reward itself. Online reward is determined from newly covered bins, as described in Section IV-B. Rarity is complemented by $U(b) = -f_b \log f_b - (1-f_b)\log(1-f_b)$, and its cluster average is $U_{\text{avg}}(C) = |C|^{-1} \sum_{b \in C} U(b)$. The entropy-shaped term is a heuristic uncertainty indicator over historical hit share, not the entropy of a binary coverage event.

Let $\mathbf{x}_b \in \{0,1\}^N$ denote the binary hit observations of bin b over N historical simulation runs, where $x_b^{(r)} = 1$ if run r covers bin b and 0 otherwise.

Let $V_C = \{b \in C \mid \text{Var}(\mathbf{x}_b) > 0\}$ denote the set of bins with non-constant hit vectors in cluster C. The redundancy and diversity scores are defined as

$$\begin{aligned} \text{Red}(C) &= \frac{2}{|V_C|(|V_C|-1)} \sum_{\substack{i,j \in V_C \\ i<j}} |\rho_{\text{P}}(\mathbf{x}_i, \mathbf{x}_j)|, \\ \text{Div}(C) &= 1 - \text{Red}(C). \end{aligned} \tag{1}$$

A high absolute correlation indicates redundant evidence, so clusters containing weakly dependent bins receive higher diversity scores. Here $\rho_{\text{P}}(\mathbf{x}_i, \mathbf{x}_j)$ is the Pearson correlation coefficient between the binary hit vectors of bins i and j. We define $\text{Red}(C) = 0$ when $|V_C| < 2$.

C. UCB Exploration and Test Scheduling

Coverage-directed scheduling is an online decision problem: the scheduler chooses a sequence before observing its coverage reward. We use the UCB rule to balance empirical reward and exploration:

$$\text{UCB}(a,t) = \bar{r}_a(t) + c\sqrt{\frac{\log(N(t)+1)}{n_a(t)+\epsilon}}, \tag{2}$$

where $\bar{r}_a(t)$ is the empirical reward, $n_a(t)$ is the number of selections of action a, $N(t)$ is the total number of selections, c controls exploration, and ϵ avoids division by zero. Section IV-B extends this rule into a two-level policy over clusters and sequences.

IV. Proposed Framework

Fig. 1 summarizes the framework. Offline clustering and scoring transform raw simulation feedback into information-aware clusters; online execution updates coverage, dynamic importance, rewards, and runtime statistics before refilling freed worker slots with two-level UCB scheduling.

A. Offline Coverage Abstraction and Prior Construction

1) Component-Aware Coverage Clustering: Bin labels are parsed into namespace, component, method, parameter, range, and structural features, distinguishing cross bins from cover-points. Component partitions exceeding a size limit are split by DBSCAN [23].

2) Information-Based Cluster Scoring: For cluster C, the score uses:

- rarity score $R(C) = \mathcal{N}(I_{\text{avg}}(C))$, computed from the mean self-information of bins in C;
- uncertainty score $E(C) = \mathcal{N}(U_{\text{avg}}(C))$, computed from the entropy-shaped function of the historical relative hit frequency;
- diversity score $D(C) = \mathcal{N}(\text{Div}(C))$, estimated as one minus average absolute correlation among bin-hit vectors;
- size-balance score $S(C) = \mathcal{N}(\log(1+|C|))$, computed from the cluster size;
- structural bonus $X(C) = \mathcal{N}(X_{\text{raw}}(C))$, assigned to cross-like and structurally complex coverage bins.

These static metrics are normalized by their positive-cluster mean:

$$\mathcal{N}(z(C)) = \frac{z(C)}{\frac{1}{|\mathcal{C}^+|}\sum_{C' \in \mathcal{C}^+} z(C')}, \tag{3}$$

where $\mathcal{C}^+ = \{C \mid z(C) > 0\}$.

The structural bonus captures bins that are likely to encode interaction behavior:

$$X_{\text{raw}}(C) = \frac{|\{b \in C \mid b \text{ is cross-like}\}|}{|C|} + \delta(C), \tag{4}$$

where $\delta(C)$ is a small residual-cluster bonus; cross-like bins contain a cross-coverage marker or template structure.

The final static cluster importance is a weighted combination:

$$W(C) = k_R R(C) + k_E E(C) + k_D D(C) + k_S S(C) + k_X X(C), \tag{5}$$

where $R(C)$, $E(C)$, $D(C)$, $S(C)$, and $X(C)$ denote normalized rarity, entropy-shaped uncertainty, diversity, size balance, and structural bonus respectively.

The weights $k_R,\ k_E,\ k_D,\ k_S,\ k_X$ are fixed for all evaluated designs, with larger value assigned to rarity and uncertainty.

3) Dynamic cluster prior Update: We define the dynamic cluster prior, which combines static importance with remaining opportunity and is refreshed after every completed simulation. Its logarithmic residual term $L_t(C)$ is distinct from the linear uncovered ratio $Q_t(C)$, which is used later in the opportunity-weighted online reward.

Let $c_t(C)$ be the number of bins in cluster C that have been covered in the accumulated coverage state before scheduling step t. We define a residual information term

$$L_t(C) = -\log\left(\frac{c_t(C)+1}{|C|+1}\right), \tag{6}$$

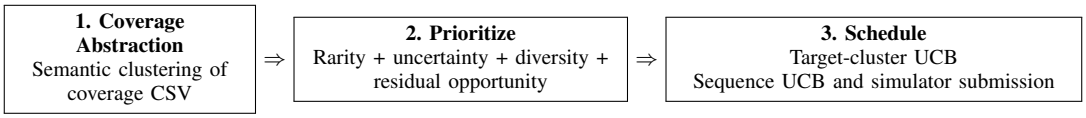


Fig. 1. Closed-loop operation of the framework. Offline abstraction converts coverage names and initial runs into semantic clusters. Information-based scoring assigns cluster priorities, and the two-level UCB scheduler selects a target cluster and a test sequence for simulation. Each completed run updates new-bin rewards, dynamic cluster prior, and runtime statistics before the freed worker is refilled and future submissions are reselected.

where add-one smoothing keeps the residual finite. It is mean-normalized as

$$\widehat{L}_t(C) = \frac{L_t(C)}{\frac{1}{|\mathcal{C}_t^+|}\sum_{C' \in \mathcal{C}_t^+} L_t(C')}, \tag{7}$$

where $\mathcal{C}_t^+ = \{C \mid L_t(C) > 0\}$.

The dynamic cluster prior is defined as

$$P_t(C) = W(C)\widehat{L}_t(C). \tag{8}$$

The two-level UCB utilizes its mean-normalized form

$$\widetilde{P}_t(C) = \frac{P_t(C)}{\frac{1}{|\mathcal{C}_t^P|}\sum_{C' \in \mathcal{C}_t^P} P_t(C')}, \tag{9}$$

where $\mathcal{C}_t^P = \{C \mid P_t(C) > 0\}$.

For example, consider a cluster containing ten bins, of which two have been covered. Its static score summarizes rarity, uncertainty, diversity, size, and structure in the initial corpus, while its residual term reflects the eight remaining bins. Their product forms the dynamic prior used by the first-level UCB. After this cluster is selected, the second-level UCB chooses a historically associated sequence, and only newly covered bins contribute to subsequent online rewards.

B. Continuous-Refill Two-Level UCB Scheduling

The test sequence scheduler consists of two UCB levels. The first UCB level selects a target cluster using $\widetilde{P}_t(C)$ and the second selects a matching test sequence. Completed simulations update feedback immediately and freed worker slots are refilled without waiting for a batch.

1) Online feedback and continuous refill: Let $\mathcal{C}$ be the set of coverage clusters and $\mathcal{S}$ be the set of available test sequences for selection. At scheduling step t, the accumulated coverage state determines whether a bin has already been covered. We define the remaining opportunity of cluster C as $u_C(t) = |\{b \in C \mid b \text{ has not been covered before } t\}|$. The corresponding runtime uncovered ratio is $Q_t(C) = u_C(t)/|C| = 1 - |\{b \in C \mid h_b(t) > 0\}|/|C|$. A cluster with $u_C(t) = 0$ is excluded for selection. The run budget is H, with at most P concurrent simulations; reselection affects only future submissions.

Because workers finish asynchronously, each submitted run retains the target cluster assigned at submission time. For a completed run r, let C_r^{sub} denote this submitted target and let $\Delta\mathcal{B}_r$ be the bins covered by run r that remain uncovered in the accumulated state immediately before processing that result. We use three feedback signals. The first is the number of newly covered bins, $\Delta_{\text{bin}}(r) = |\Delta\mathcal{B}_r|$. The second is the target-cluster gain,

$$\Delta_{\text{tar}}(r) = |\{b \in \Delta\mathcal{B}_r \mid b \in C_r^{\text{sub}}\}|, \tag{10}$$

Let $\phi : \mathcal{B} \to \mathcal{C}$ map each bin to its cluster. The opportunity-weighted gain is

$$\Delta_{\text{opp}}(r) = \sum_{b \in \Delta\mathcal{B}_r} \left(1 + Q_{r^-}(\phi(b))\right), \tag{11}$$

where r^- is the state before incorporating run r; the ratio rewards new bins in less-complete clusters.

The three signals serve different purposes: Δ_{tar} updates cluster reward, Δ_{bin} updates sequence reward and progress, and Δ_{opp} triggers reselection.

2) First-level UCB for Cluster Selection: The first-level policy chooses the next coverage cluster to target. For each unsaturated cluster C, the scheduler estimates its exploitation value from the remaining uncovered bins and the historical gain observed when C was selected:

$$\Gamma_C(t) = u_C(t)\left(\eta + \bar{g}_C(t)\right), \tag{12}$$

where $\bar{g}_C(t)$ is the empirical mean of target-cluster gains (10) from completed runs whose submitted target was C. η is a small value preventing clusters with no observed gain from being assigned zero exploitation value.

To make the online scheduler sensitive to the information content estimated from both the initial corpus and the current coverage state, the first-level UCB rule uses the normalized dynamic cluster prior $\widetilde{P}_t(C)$ from Section IV-A3. This value is refreshed after each completed simulation, after the accumulated coverage map has been updated. Therefore, a cluster can lose priority as it becomes covered, while clusters that remain under-covered and information-rich continue to receive additional pressure.

The cluster-level UCB score consists of a cluster-prior-weighted exploitation term and a second term for exploration, and is denoted as

$$U_C(t) = \Gamma_C(t)\left(1 + \beta(\widetilde{P}_t(C) - 1)\right) + \lambda_C(\rho_t)\sqrt{\frac{\log(N_C(t)+1)}{n_C(t)+1}}, \tag{13}$$

where β is a tunable coefficient controlling the strength of the mean-normalized dynamic cluster prior. Here $n_C(t)$ is the number of times cluster C has been selected, $N_C(t) = \sum_{C'} n_{C'}(t)$ is the total number of cluster selections, and $\rho_t = |\{b \in \mathcal{B} \mid b \text{ has been covered before } t\}|/|\mathcal{B}|$ is the global coverage progress. The cluster-level exploration coefficient is defined as a non-increasing coverage-progress schedule

$\lambda_C(\rho_t) = \lambda_C^{\max} - (\lambda_C^{\max} - \lambda_C^{\min})\rho_t$, where $\lambda_C^{\max} \geq \lambda_C^{\min} > 0$. In (13), the first term favors clusters with substantial remaining coverage opportunity, higher observed historical gain, and larger dynamic cluster priors; the second term allows clusters with limited observations to remain competitive even when their historical reward is low.

The implementation instantiates this monotonic schedule with a piecewise progress-aware policy.

When no directed fallback is invoked, the first-level decision is $C_t = \arg\max_{C \in \mathcal{C},\, u_C(t)>0} U_C(t)$.

3) Second-Level UCB for Sequence Selection: After selecting the target cluster C_t, the scheduler uses the available coverage vectors to construct a target-conditioned sequence domain. A sequence is regarded as empirically relevant to the current target if its historical executions have exercised at least one bin in C_t: $\mathcal{S}_{C_t} = \{s \in \mathcal{S} \mid H(s, C_t) = 1\}$, where $H(s, C_t) = 1$ means that a historical execution of sequence s covered at least one bin in C_t; otherwise, $H(s, C_t) = 0$. When the available data provide no such evidence for the selected cluster, the domain defaults to the complete sequence set, i.e., $\mathcal{S}_{C_t} = \mathcal{S}$, so that the scheduling decision remains well defined. This construction treats observed target reachability as a practical relevance prior and concentrates the second-level comparison on sequences with empirical support for the selected coverage region.

For each candidate sequence s, the scheduler estimates its target-conditioned incremental gain as

$$\bar{g}_{s,C_t}(t) = \frac{1}{|\mathcal{R}_s(t)|} \sum_{r \in \mathcal{R}_s(t)} |\Delta\mathcal{B}_r \cap C_t|\,, \tag{14}$$

where $\mathcal{R}_s(t)$ is the set of completed adaptive runs generated by sequence s before decision step t, and $\Delta\mathcal{B}_r$ contains the bins that remain uncovered in the accumulated global coverage state immediately before the completion result of run r is processed. Hence, only newly discovered bins within the selected target cluster contribute to the exploitation estimate; repeated hits of previously covered bins receive no additional reward. We set $\bar{g}_{s,C_t}(t) = 0$ when $\mathcal{R}_s(t)$ is empty. The sequence-level UCB score is

$$U_s(t \mid C_t) = \bar{g}_{s,C_t}(t) + \lambda_S(\rho_t)\sqrt{\frac{\log(N_S(t)+1)}{n_s(t)+\epsilon}}, \tag{15}$$

where $n_s(t) = |\mathcal{R}_s(t)|$ is the number of completed adaptive executions of sequence s, $N_S(t) = \sum_{s'} n_{s'}(t)$ is the total number of completed adaptive executions, and ϵ is a small constant for numerical stability. Initial coverage vectors may provide reachability evidence for constructing $\mathcal{S}_{C_t}$, but they do not contribute to $\mathcal{R}_s(t)$ or the incremental reward. Sequences with $n_s(t) = 0$ are handled as cold-start actions.

Similarly, the sequence-level exploration coefficient is defined as a

This schedule controls exploration among candidate sequences conditioned on the selected target cluster. Exploration is performed within $\mathcal{S}_{C_t}$; the full sequence space is used only when no sequence has observed support for the selected target. non-increasing coverage-progress schedule $\lambda_S(\rho_t) = \lambda_S^{\max} - (\lambda_S^{\max} - \lambda_S^{\min})\rho_t$, where $\lambda_S^{\max} \geq \lambda_S^{\min} > 0$.

TABLE I
MAIN SCORING AND SCHEDULING PARAMETERS.

Parameter	Value	Role
k_R, k_E, k_D	$0.40, 0.25, 0.20$	static score weights
k_S, k_X	$0.10, 0.05$	
β	0.20	dynamic-prior strength
η	0.05	cluster cold-start prior
ϵ	10^{-8}	sequence UCB stability
$\lambda_C^{\min}, \lambda_C^{\max}$	$1.2, 5.0$	cluster exploration schedule
$\lambda_S^{\min}, \lambda_S^{\max}$	$1.0, 3.0$	sequence exploration schedule
$\alpha_{\min}, \alpha_{\max}$	$0.10, 0.50$	runtime penalty range

*a) **Runtime-aware sequence scoring.**:* Simulation runtime is an important practical cost in verification. A sequence that occasionally produces new coverage but consumes much longer simulation time may be less attractive than a shorter sequence with comparable expected gain. The scheduler therefore applies a runtime-aware penalty after computing sequence-level UCB. Let $T_s(t)$ be the empirical average runtime of sequence s, and define the normalized runtime as $\tilde{T}_s(t) = T_s(t)/\max_{s' \in \mathcal{S}} T_{s'}(t)$. If no positive runtime observation is available, the normalized penalties are set to zero. The penalty coefficient grows with coverage progress as $\alpha(\rho_t) = \alpha_{\min} + (\alpha_{\max} - \alpha_{\min})\rho_t$, where $\alpha_{\min} < \alpha_{\max}$ so that runtime cost is penalized more strongly near coverage saturation.

$$A_s(t \mid C_t) = U_s(t \mid C_t)\left(1 - \alpha(\rho_t)\tilde{T}_s(t)\right). \tag{16}$$

The scheduler selects the sequence with the largest runtime-aware action score $s_t = \arg\max_{s \in \mathcal{S}_{C_t}} A_s(t \mid C_t)$. This separate multiplicative correction preserves $U_s(t \mid C_t)$ as indicator for coverage potential while making $A_s(t \mid C_t)$ reflect practical execution cost. A long-running sequence can still be selected, but it must provide higher expected coverage value to compensate for its runtime.

*b) **Closed-loop submission.**:* The selected pair (C_t, s_t) is then submitted to the simulator, forming a closed loop in which coverage information defines value, UCB decides where and how to spend simulation effort, and runtime-aware scoring accounts for cost.

V. EXPERIMENTAL METHODOLOGY

A. Design Under Test

Our method is implemented as an automated simulation-feedback flow around a commercial simulator and functional-coverage analysis tool. Reports are parsed into a bin-by-run coverage table, raw bin labels are grouped into information-scored clusters, and completed simulations update coverage rewards, runtime statistics, dynamic cluster prior, and the next UCB decision. We evaluate it on OpenTitan AON Timer and a large industrial IP with 19,919 bins.

Due to industrial confidentiality restrictions, only a subset of the regression test cases was available for evaluation. Historical runs of this accessible subset reached approximately

TABLE II
AON TIMER ABLATION RESULTS.

Method	Cov. (%)	Runtime (s)	Wall time (s)
Full	75.33	2928.25	4764.13
No Incremental Refill	75.33	3009.35	6160.74
No Runtime Penalty	75.33	2949.53	4943.13
Random Selection	16.67	15355.71	4450.20
Sequence UCB Only	52.15	9594.78	4698.70
Cluster UCB Only	27.17	47728.17	6629.73

65% raw functional coverage, which we use as an empirical reference level $\kappa = 0.65$. We therefore also report the normalized coverage, i.e.,

$$\mathrm{Cov}_{\mathrm{ref}}(t) = \frac{\mathrm{Cov}_{\mathrm{raw}}(t)}{\kappa}. \tag{17}$$

Note that this normalization is not intended to represent a formally established reachability bound.

All methods use the same initial coverage corpus, total simulation budget, parallelism, and random seed. The evaluation uses a logical batch size of 10, 100 logical batches, 1000 submitted simulations in total, 10 parallel workers, and a random seed.

B. Compared Methods and Metrics

Our method, denoted as **Full**, combines continuous refill, cluster-level UCB, sequence-level UCB, and runtime-aware scoring. We compare it with Random Selection and ablations that remove individual scheduling components. Accumulated runtime is the sum of simulator runtimes across completed runs, whereas wall time is the end-to-end elapsed time including parallel execution, coverage-report generation, and scheduling overhead. We report final coverage, accumulated runtime, and wall time.

VI. RESULTS

A. Overall Comparison

Table II summarizes the ablation results. The full method reaches 75.33% coverage, while random selection reaches only 16.67% despite consuming more accumulated simulation runtime.

Fig. 2 shows that Full reaches 75.33% coverage with much less runtime than Random Selection.

Full achieves a 4.22× higher final-coverage-to-wall-clock ratio than Random Selection.

B. Large Industrial IP Case Study

We further evaluate the scheduler on the large industrial IP under the same fixed regression pool. Raw functional coverage is the primary metric; coverage normalized by the empirical reference level κ is reported as a supplementary measure.

Table III compares closure results from the same initial coverage state. No Info removes the information-derived cluster-prior factor from target selection while retaining continuous refill, both UCB levels, and runtime-aware scoring. Full reaches 60.16% raw coverage, compared with 57.87% for No Info and 55.09% for Random Selection, and uses 3.2% less accumulated runtime than No Info. Because No Info already reaches 57.87% against the 65% empirical reference level, the 2.29 percentage-point raw gain of Full closes 32.1% of the remaining reference-level gap. The corresponding normalized values are reported only as supplementary results in Table III.

These results suggest that the dynamic cluster prior provides an additional benefit beyond the two-level UCB scheduler in this late-stage industrial regression setting.

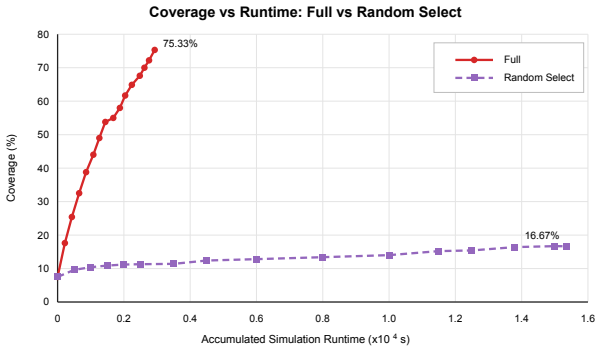


Fig. 2. AON Timer coverage growth.

TABLE III
COVERAGE RESULT ON THE LARGE INDUSTRIAL IP.

Metric	Random	No Info	Full
Final cov. (raw)	55.09%	57.87%	60.16%
Final cov. (normalized)	84.75%	89.03%	92.55%
Normalized gain	15.52 pp	19.80 pp	23.32 pp
Runtime (10^6s)	1.91	1.86	1.80

C. Ablation Analysis

In the AON Timer experiment, the complete two-level scheduler outperforms both single-level ablations. Sequence UCB Only reaches 52.15%, indicating that sequence feedback helps, but it remains 23.18 percentage points below Full because it lacks explicit target-cluster selection. Cluster UCB Only reaches 27.17% and consumes substantially higher runtime, indicating that cluster prioritization alone is insufficient in this experiment.

Continuous refill mainly improves wall-clock time rather than final coverage. Full and No Incremental Refill both reach 75.33% final coverage, but Full reduces wall-clock time from 6160.74s to 4764.13s, a 22.7% reduction. This is consistent with the motivation of asynchronous refill: when simulations have heterogeneous runtimes, waiting for an entire batch delays the use of feedback from faster tests and leaves worker slots underutilized.

Runtime-aware scoring provides an additional efficiency improvement. Full and No Runtime Penalty reach the same final coverage of 75.33%, but Full reduces wall-clock time from 4943.13s to 4764.13s, a 3.6% reduction. This is consistent with the runtime-aware score in Section IV-B: long-running sequences can still be selected, but they must provide stronger expected coverage value.

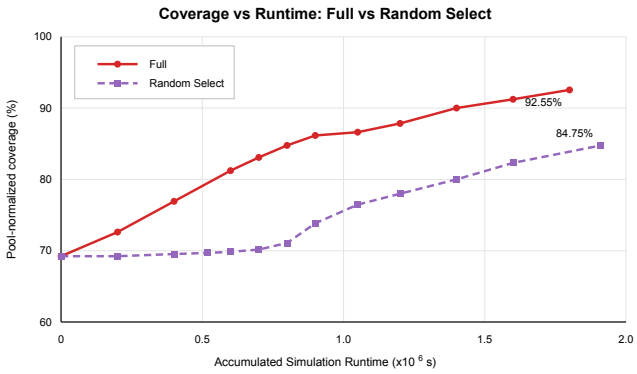


Fig. 3. Normalized industrial-IP coverage closure process.

Together, these results show that dynamic-information-prior two-level UCB drives coverage growth, while continuous refill and runtime-aware scoring improve practical simulation efficiency.

Limitations. The evaluation uses one random seed and two designs, so the results are case-study evidence rather than statistical proof. Weight sensitivity, multi-seed tests, industrial ablations, and common-infrastructure comparisons remain future work.

VII. Conclusion

This paper proposes an entropy-aware and UCB-driven coverage closure framework for hardware verification. By combining dynamic cluster prioritization, two-level UCB selection, continuous refill, and runtime-aware scoring, the method improves coverage growth and simulation efficiency over random selection and evaluated ablations on OpenTitan AON Timer and an industrial IP.

Acknowledgment

This work was supported in part by the 2025 Basic and Applied Basic Research Program of Guangzhou Municipal Science and Technology Bureau (Young Doctor "Qihang" Project, Task No. 2025A04J4001), and in part by Shenzhen Bureau of Science and Technology Research under Grant No. KJZD20240903102708012.

References

[1] F. Haedicke, H. M. Le, D. Grosse, and R. Drechsler, "CRAVE: An advanced constrained RAndom verification environment for SystemC," in *2012 International Symposium on System on Chip*. Piscataway, NJ, USA: IEEE, 2012, pp. 1–7.

[2] S. Fine and A. Ziv, "Coverage directed test generation for functional verification using bayesian networks," in *Proceedings of the 40th Annual Design Automation Conference*. New York, NY, USA: ACM, 2003, pp. 286–291.

[3] C. Ioannides and K. I. Eder, "Coverage-directed test generation automated by machine learning: A review," *ACM Transactions on Design Automation of Electronic Systems*, vol. 17, no. 1, pp. 1–21, 2012.

[4] C. Liu, J. P. Parlapalli, D. K. Houngninou, M. Quinn, A. Tyagi, and J. Hu, "Improving last-mile coverage in functional verification," in *2025 ACM/IEEE 7th Symposium on Machine Learning for CAD*. Piscataway, NJ, USA: IEEE, 2025, pp. 1–7.

[5] C. E. Shannon, "A mathematical theory of communication," *The Bell System Technical Journal*, vol. 27, no. 3, pp. 379–423, 1948.

[6] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Machine Learning*, vol. 47, no. 2–3, pp. 235–256, 2002.

[7] A. Jayasena and P. Mishra, "Directed test generation for hardware validation: A survey," *ACM Computing Surveys*, vol. 56, no. 5, pp. 1–36, 2024.

[8] K. A. Ismail and M. A. Abd El Ghany, "Survey on machine learning algorithms enhancing the functional verification process," *Electronics*, vol. 10, no. 21, p. 2688, 2021.

[9] O. Guzey, L.-C. Wang, J. R. Levitt, and H. Foster, "Functional test selection based on unsupervised support vector analysis," in *Proceedings of the 45th Annual Design Automation Conference*. New York, NY, USA: ACM, 2008, pp. 262–267.

[10] ——, "Increasing the efficiency of simulation-based functional verification through unsupervised support vector analysis," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 29, no. 1, pp. 138–148, 2010.

[11] W. Chen, N. Sumikawa, L.-C. Wang, J. Bhadra, X. Feng, and M. S. Abadir, "Novel test detection to improve simulation efficiency: A commercial experiment," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. New York, NY, USA: ACM, 2012, pp. 101–108.

[12] G. Parthasarathy, A. Rushdi, P. Choudhary, S. Nanda, M. Evans, H. Gunasekara, and S. Rajakumar, "RTL regression test selection using machine learning," in *2022 27th Asia and South Pacific Design Automation Conference*. Piscataway, NJ, USA: IEEE, 2022, pp. 281–287.

[13] R. Liang, N. Pinckney, Y. Chai, H. Ren, and B. Khailany, "Late breaking results: Test selection for RTL coverage by unsupervised learning from fast functional simulation," in *2023 60th ACM/IEEE Design Automation Conference*. Piscataway, NJ, USA: IEEE, 2023, pp. 1–2.

[14] F. Wang, H. Zhu, P. Popli, Y. Xiao, P. Bogdan, and S. Nazarian, "Accelerating coverage directed test generation for functional verification: A neural network-based framework," in *Proceedings of the Great Lakes Symposium on VLSI*. New York, NY, USA: ACM, 2018, pp. 207–212.

[15] S. Vasudevan, W. Jiang, D. Bieber, R. Singh, H. Shojaei, R. Ho, and C. Sutton, "Learning semantic representations to verify hardware designs," *Advances in Neural Information Processing Systems*, vol. 34, pp. 23 491–23 504, 2021.

[16] Y. Lu, C. Bai, Y. Zhao, Z. Zheng, Y. Lyu, M. Liu, and B. Yu, "DeepVerifier: Learning to update test sequences for coverage-guided verification," *ACM Transactions on Design Automation of Electronic Systems*, vol. 31, no. 4, pp. 1–23, 2026.

[17] Q. Huang, H. Shojaei, F. Zyda, A. Nazi, S. Vasudevan, S. Chatterjee, and R. Ho, "Faster coverage convergence with automatic test parameter tuning in constrained random verification," in *Workshop on Languages, Tools, and Techniques for Accelerator Design (LATTE)*, 2021.

[18] N. Masamba, K. Eder, and T. Blackmore, "Supervised learning for coverage-directed test selection in simulation-based verification," in *2022 IEEE International Conference on Artificial Intelligence Testing (AITest)*, 2022, pp. 19–25.

[19] K. Laeufer, J. Koenig, D. Kim, J. Bachrach, and K. Sen, "RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs," in *Proceedings of the IEEE/ACM International Conference on Computer-Aided Design*. New York, NY, USA: ACM, 2018, pp. 1–8.

[20] S. Canakci, L. Delshadtehrani, F. Eris, M. B. Taylor, M. Egele, and A. Joshi, "DirectFuzz: Automated test generation for RTL designs using directed graybox fuzzing," in *2021 58th ACM/IEEE Design Automation Conference*. Piscataway, NJ, USA: IEEE, 2021, pp. 529–534.

[21] V. Gohil, R. Kande, C. Chen, A.-R. Sadeghi, and J. Rajendran, "MABFuzz: Multi-armed bandit algorithms for fuzzing processors," in *2024 Design, Automation & Test in Europe Conference & Exhibition*. Piscataway, NJ, USA: IEEE, 2024, pp. 1–6.

[22] G. Dimitrakopoulos, E. Kallitsounakis, Z. Takakis, A. Stefanidis, and C. Nicopoulos, "Multi-armed bandits for autonomous test application in RISC-V processor verification," in *2023 12th International Conference on Modern Circuits and Systems Technologies (MOCAST)*, 2023, pp. 1–5.

[23] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu, "A density-based algorithm for discovering clusters in large spatial databases with noise," in *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. Portland, OR, USA: AAAI Press, 1996, pp. 226–231.